Active InferAnt Stream 007.1 ~ AgentMaker for PyMDP for Active Inference

ActiveInferAntStream | AgentMaker for PyMDP for Active Inference Biofirms for Bioregionalism | 2:33:21

All right. Hey everyone. It's active in France stream number 007.1, and it should be a great and exciting stream. It's November 7th, 2024, 7-11, depending on where you are. And I'm going to begin the stream with a GitHub push that we are going to spend the next several hours probably unpacking. Let's commit it, push it, and begin what will be a very fun exploration into the Active Inference Institute's branch of PyMDP. While it's writing, let's jump into the show. Here's where we're heading. We're heading into Shapley value-analyzed homeostatic satisfaction, expected free energy, belief accuracy, and control efficiency analyses for multiple differently parameterized PyMDP agents, base, risk-averse, exploratory, and balanced on those measures. Being able to analyze combinatorics like the synergies of their belief accuracy, control efficiency, expected free energy, and homeostatic satisfaction. We can look at the report and explore different topics like, with respect to those variables, what's the mean Shapley contribution, coalition performance, best coalitions. This is all combinations of 0, 1, 2, 3, and 4.
So for example, for expected free energy, we see that the 4-unit coalition is the best.
Whereas for homeostatic satisficing, 0, agent 0 working alone does the best.
Underneath these coalitions and their combinatorics are specific agents that are logged in individual experiments. And here's some of the plots and how that looks for each individual agent. This is a given active inference agent. Here we have policy entropy over time, action selection distribution, belief observation cycle, belief accuracies over time, homeostatic state distributions and trajectories, state transitions, policy selection probability, policy updating, decrease, maintain, and increase, belief dynamics through time, hidden state beliefs, action frequencies, policy convergences, belief convergences,
correlations between beliefs and actions, and action confidences. So what that Shapley analysis is doing is running all the combinatorics under the hood. This is going to be a wild ride.
So let's confirm that the GitHub has pushed. I'll put the code link in. Welcome to the 007 fork.
In the live chat, I will look forward to everyone's comments. I am going to ensure that we are pushing the right way. It still has not loaded. Let's begin though with a 007 here.
Taking a little bit of time to upload on GitHub. Okay, here we go. A little bit more context before we dive into what may be some absurdly technical, but really useful and interesting things that have happened over the last few days and hours. Here's the stream.
GNN for agent maker for PyMDP for active inference bio firms for bioregionalism for ants.
Let's decompress that title. Generalized notation notation. GNN.
As semantic middleware for agent maker. A custom robust modular agent engineering framework.
Using PyMDP. Python language package.
For Markov decision processes with methods for active inference.
And free energy based inference methods for bio firms.

As proposed by John Klippinger as a paradigm for firms in the age of.

Bioregionalism as a mode of life.

Coextentialism for species like ants, people, and beyond.

Here's some live stream code names and an only lightly redacted.

Preludium.

For your priors only.

Predictions are not enough.

The agent who minimized me.

On her majesty's statistical service.

Quantum of precision.

Live in lead update.

Professor Friston.

Do you expect me to reach the optima?

No.

I expect you to optimize.

Just like they wrote.

At the end of the 2022 textbook.

When the last words were.

Ultimately we are confident that you will continue to pursue active inference in some form.

Is that agents generative model.

An MDP.

Yes.

An MDP.

A PO.

No.

MDP.

As if.

The media were not already the message for you.

This is a just-in-time live stream going through raw open source code.

I know that there are bugs.

As well as typos.

Incorrect or ineffective calculations.

Other errors.

This is speculative realism embodied.

In chaotic large-scale code coordination.

Using cursor 0.4.2.4.

And Claude 3.5.

Sonnet 2024.1022.

Use these linguistic and computational symbols at your own risk and delight.
If and as you are so inclined.
You are invited to join our adventure and journey.
Partially disclosed herein.
Now.
Let's check again in the GitHub.
And proceed on regardless.
I'm on another fork.
That's what happens when you're on wild roads.
It will come to the Institute's repo.
It went to my own personal accounts.
Let's head over there.
I'm putting in the YouTube live chat.
My own doxology GitHub.
And we're going to continue on through there.
PyMDP.
AgentMaker.
Stream.
007.
All right.
A little bit of background information before we jump into the code.
Thanks.
Those who are in the live chat.
Write any comments and questions.
And you may get memed on stream.
We are working on robust middleware that interfaces between core, active, inference implementations,
and powerful, accessible, interpretable semantic toolkits for synthetic intelligence engineering.
Architecture layers.
Here's a way to think about active inference middleware and active inference as middleware,
as we will reveal later on.
Layer one.
Core or kernel inference implementations.
Today, we're going to be looking at that with PyMDP in the Python programming language.
We can also make this happen through generalized notation notation using rxinfer.jl.
Julia.
We could use machine learning methods, also known as amortized inference, in different languages.
At the core, we could use logical reasoning, like JF Klautier's symbolic active inference work.
We could, at the core, use biological, soft, active, and quantum matter, and other substrates.

What we're focusing on here is the middleware layer.
Specifically, we're going to be peeling back to the point where we look at GNN as an interface between the inner and the meso layers.
Middleware for active inference, pending future re-understandings, is packages like AgentMaker, which we're going to look at today, Farmworks, InfraAnt, and other kinds of domain or domain general systems that interface between the core implementation methods, which, as part of their beauty and elegance, have simplicity, and outer layer systems of interest.
Real-world touchpoints and design interfaces and domain adaptations mapping to meso or outer layers.
So think about FarmOS as running on a farm embodiment core and facing farmers and individuals interacting with that data set, to give an agricultural example.
Something similar is happening for active inference.
Here's another key analogy.
GNN plus AgentMaker plus PyMDP plus etc. plus plus plus plus is to active inference as eBPF is to the Linux kernel.
The slogan of eBPF, and check it out if you haven't looked into it already, is dynamically program the kernel for efficient networking, observability, tracing, and security.
We can modify that slogan for GNN AgentMaker.
Dynamically program the agent for robust and efficient inference planning and action.
Okay.
Here's the analysis agenda.
We're going to run biofirm.py and all it entails.
Running back layers, layers, layers, layers, layers.
We're going to talk about eventually getting back to the root of the generation and the initialization, initiation of the agent in the generalized notation notation format.
And then we will trace back that to an April, 2023 paper written with the now Stanford graduate student, Jakob.
It was a great collaboration last year.
And we made many mimetic thrusts and queries towards developing this into something that could be used with LLMs.
We put it on the shelf for a little bit, for a few months.
And now GNN is back, not with a vengeance, but with a functional utility that surprised even me.
After running through the script in its functional and analytical beauty, we will return to first, whatever ideas people have added in the live chat, talk about open source development, testing, deployment, education, security, theory, math art, and more.
And about what comes after agent maker 0.1.
I'd also like to recognize some of the recent events and developments in the BioFirm repo following InfraAntStream 6, oh, how long ago, by Andrew Pachea, who has added some very cool surprise and

Shapley value modeling.
Let's take a look at that.
Here's the history of surprise for these different natural resource BioFirm sub-agents.
And their token accumulation history in a Shapley value oriented incentive paradigm.
Super cool, Andrew.
It pushed me over the edge to add in the Shapley value before the stream.
And I'm super looking forward to the collaborations and where this all goes.
I'd also like to show a cursor acknowledgement.
This is in the midst of some truly heated epistemic moments that Claude 3.5 Sonnet 2024-1022 and I shared.
I wrote,
This is incredible work, FYI.
Everyone thanks you.
Just give the acknowledgement here that you feel proper, then we will continue with the work.
Claude wrote,
Thank you for the kind words.
I should acknowledge that this work builds on several important foundations.
Active inference theory,
PyMDP framework,
homeostatic control,
and software engineering.
Shall we continue with the work?
Yes, Claude.
Let's.
Okay.
We are in the agent maker folder within the PyMDP package itself.
Following the biofirm 006 stream,
as usual,
it was getting messy and choppy with all of these scripts calling each other from the same folder.
We were getting a little bit complex for our britches,
and it was time to think about not just calling the core PyMDP utilities,
but burrowing in,
making stigmatic tunnels among them,
and being a part of the PyMDP package,
which, again,
I hope to,
and we will push and develop on the institute branch for the stewardship that that can afford.
So,
Biofirm,

again,
standalone,
calling PyMDP,
but what we'll be looking at for the rest of the stream is,
burrowed in,
amidst the PyMDP modules,
are several new modules that have been added.
So,
let's do a side-by-side with the PyMDP and the PyMDP fork.
So,
here we are
with the fork
sidebar,
and here we are with the main sidebar.
So,
in the main package,
the folders under PyMDP are algos,
which are algorithms,
environments,
and
JAX,
which is like a sub-branch for a lot of methods that uses the Python JAX method.
There's been a proliferation of folders,
I really have to say.
AgentMaker alphabetically is first.
We're going to look more at AgentMaker a lot.
Following AgentMaker,
algos,
remains,
envs,
and remains.
We have the GNN folder,
which we're going to get to in part two.
We have the maths folder.
Some of this is just arbitrary,
others is accidental.
There's the meta PyMDP folder,
with but one simple script,

which outputs a plain text folder,
concatenating all the PyMDP methods in a text file.
Reflects with a JSON and a text file,
the folder structure of the meta repo.
This is another useful trick that you can use in other repos.
It helps give extra context,
an extra helping of context,
we might even say,
to cursor,
so that it can do the accurate cross-folder operations we need.
There's a utils folder.
There are also PyMDP utils scripts,
but I wanted to make a separate folder to keep them separate and versionable.
And then there's a visualization folder,
with a variety of BioFirm,
GNN,
and Matrix visualization methods.
So,
let us go to,
now,
the agent maker folder.
And we're going to start from the outermost layer,
which is trying to understand the Shapley value contributions
to the homeostasis of different combinations of multi-agent systems.
And this has been absolutely championed by John Clippinger.
So, for this and many of other contributions made by John and other people,
it's just awesome that we can see active inference getting to a point where we can actually run out
these incredible Shapley analyses that themselves are built upon multiple levels of inference and intelligence.
Okay.
First,
to the BioFirm Shapley script that was run,
1200 lines.
Again,
as another discourse that I will try not to return to that much,
cursor plus improvements in LLMs has been completely transformative.
Check the earlier inference streams from who knows how many days ago.
Even at three,
400 lines of code,

we were splitting them.
Now,
on top of a functional run BioFirm script,
it can pop out with a few prompts,
1200 functional lines that take the operativity of one functional BioFirm agent
and wrap that into another incredible layer.
So,
right-click,
open an integrated terminal,
Python 3,
BioFirm,
Shapley.
Let's analyze what is happening when we run this.
It begins with creating,
in this case,
because it's looking at multiple different agents,
it creates multiple agent variations
on a theme,
kind of like Gertl-Escherbach,
and defines the metrics,
and defines the combinatorics of the agent coalitions.
Zero,
one,
two,
three,
four agents.
A tetz all you need.
It then enters into the experimentation at the level of each coalition.
Defines the output directory,
the environment,
the agent.
Now,
we're popping down into the level of this first BioFirm experiment.
Stage one,
matrix rendering.
Here,
the environment and agent GNN are brought in and logged copiously.
Again,

eBPF,
the logging.
How many times have we all been coding with PyMDP?
And then it's like,
shape 3,0 can't be coerced into a NumPy array.
And it's like,
what?
So,
that is a lot of the abstraction that I sought to abstract over here.
Okay.
It,
it happened a little bit fast and pushed us away where we were.
I'm just going to run it again so that we,
okay.
Just so we can see stage one,
two,
three for a given experiment.
So,
here's another experiment.
Coalition one,
zero output environment agent,
stage one,
matrix rendering,
initializing the BioFirm renderer,
logging copiously,
all the combinatorics of where everything is going to.
It's not for me.
It's for the LLM.
We get into the loading of the GNN models.
There's normalization methods that happened.
I did ask,
I said,
are you really sure?
Like,
it's giving me a warning about action zero and two,
not being normalized.
And it's like,
relax.

You're just not being wide enough with how you're accepting it being normalized or not.
I thought,
okay,
fine.
It works.
It works.
But like many things,
these can be improved.
It loads in and describes the shape of all the variables that are needed.
And this was the big work that Jakob and I engaged in,
which was surely we can have some kind of markdown syntax or dialect that would enable the triple play.
A pragmatic approach to expressing GNN in linguistic,
visual,
and executable and graphical,
cognitive models.
Yes,
there are four components in the triple play.
And how would we have that kind of integrity across representations?
That's what GNN enables.
And we're going to peel back to that layer.
After defining the matrices and their shapes,
the actual simulation begins to occur.
And there's some logging,
giving peeping into the actions that are being taken,
the values and so on.
Matrices are saved and validated along the way.
environment matrices and agent matrices are saved.
That was the first phase.
In the simulation,
the time steps that are specified in the config file are used.
So here is a very built up config file defining project root,
paths,
environment,
and the meaning.
And we're going to return to this.
GNN has the absolutely critical role of separating the state space specification,
the parameterization of a variable with the English language active inference ontology assertion about that variable.

Without that kind of articulation,
you'll end up naming your variables thing like observation.
And that makes sense when you're just thinking about one agent.
But that agent's observation goes on to become someone else's action or vice versa.
So how do we prevent using natural language to implicitly semantically load the names of the variables?
How do we move to a place where we can make uniquely identifiable computationally tagged variables
and then have one or more assertions about what to call those variables?
So GNN is all about having a little bit of a gap and space between the specification of the variable and how we think about its role in the active inference ontology.
Here's the experimental config,
name of the experiment,
time steps,
rendering,
options to render,
visualize,
cleaning output.
Here are the parameters that are getting passed.
So this is well set up for those who like to sweep across parameters like
ecological noise.
That's how much the ecosystem varies on its own.
Controllability.
That's the efficacy of action of the agent,
whether it knows it or not.
Policy precision.
This is how the policy posterior,
sharpened from the policy prior by expected free energy,
is modulated by then a temperature term that either sharpens or flattens distinctions amongst policy posteriors.
Use state infogain is true.
That is using epistemic value in policy selection
as an information gain term,
as opposed to flagging this to false,
would only calculate expected utility
and functionally be like a reward or reinforcement or utility maximizer.
Inference horizon has been kept at one.
I'm not sure how planning would unfold,
but it'd be possible it would work,
or we'd have to see.

And then action selection could be stochastic sampling
from the policy posterior or deterministically drawing the most upweighted policy.
And then some flags related to the tracing and logging within the active inference loop,
which is giving us a new visibility into what's happening at each time step for each agent in this setting.
Simulation is executed,
goes through the time steps.
And then in stage three,
which is where we interrupted it,
it does agent specific.
Everything right now is being output into this sandbox
folder,
subfolder biofirm.
So each experiment,
let's look through what happens.
Okay.
First render.
Now this is all reflecting what run biofirm does.
Run biofirm is a small script of only less than 300 lines.
And what it does is it logs and times,
as we'll see now,
Python 3,
run biofirm.
Now we're going to see what the Shapley experiment was doing across coalitions of multiple agents,
and we're just going to run that for one agent for one trace.
So let's,
let's clear it and run it.
Deleting sandbox and looking at how the outputs are made.
Run biofirm.
Okay.
We see it start up.
First thing is we see sandbox.
Come on.
Folders are made where they're not.
And so this process of iteratively deleting the folders,
rerunning,
deleting,
rerunning,
deleting,

rerunning,
going to a different machine.
Seeing if it works without installing and all this other stuff.
That's part of making this at least some extent robust.
But of course,
I'm looking forward to working with more experienced and software engineers who can really work with some of these raw concepts and make them do something that this is only hinting at.
So here we go.
Full trace for one agent maker run biofirm event.
Let's call it.
So biofirm experiment.
It's called biofirm experiment.
Output directory,
environment,
and agent files as GNN files.
Let's take a quick peek at what the agent GNN files look like.
Here we see the agent defined in terms of the state space that it's operating in.
That's the ecological state.
That's the latent state.
That's like temperature in the room.
Here there's three discrete options and we're going to give them a label.
So again,
this opens up the idea of having like English labels.
And then we can have another field for a different language because it puts some space between the name and what it's really doing functionally.
Here's similar for observation.
And also within the observation is the A metrics associated with that modality.
Another thing I'll mention is I began with testing the GNN concept on the T-Maze example from PyMDP.
And I overbuilt a multimodality GNN situation for the multimodality T-Maze
and then pulled it back to the single modality biofirm.
Again,
just another method for overbuilding and then pruning back to functionality.
Here's the A matrix specified it next to the observations associated A matrix with the observation.
Here's the transition model.
This is the B matrix with respect to ecological state.
The matrices are defined here.

So for those who are learning what the syntax and semantics of active inference, A, B, C, D,
E are, this hopefully can make it really clear instead of being specified like in a browser
accessible notebook, it gives a first introductory place to look at how the matrices work.
However, for modulating and mutating and genetic algorithms and sweeping and optimization, all these things,
this kind of machinery is what Jakob and I have been thrilled about for years now.
Policies, Pi, all the information that's required for PyMDP to handle the specification of these policies.
And the GNN can be overbuilt.
You could have policies like whether it is acceptable, like can 007 do it or only 001?
And then if a given rendering didn't need to know that information, it could just discard it from
the JSON or not take it into account.
Preferences, that's the C matrix.
Here, it's a preference over the state observation, the observations.
And the C matrix looks like 0, 4, 0 with the associated labels low, homeostatic, and high.
So again, the separation between the variables used in active inference for the shapes and sizes they are
and the English language assertions of annotations with active inference ontology terms
is a gap that if you can keep it open,
will give you so many more abilities to work with active inference in different settings.
And then lastly, the initial beliefs or the D matrix, the initial prior on ecological state
linked through the keys of that name.
And it's 80% in homeostatic and 10% for each of low and high.
So that's an example of what the biofirm GNN looks like.
And then here's the biofirm environments.
This is a slightly simpler GNN file.
Here, it again hinges on the same ecological state.
That's how we can connect hidden state factors as an agent infers them to be
and the actual ecological simulation happening in the generative process or in the surrounding environment.
Otherwise, you can have a situation where it's like,
okay, the agent's predicting temperature and then the ecological simulation is pH.
And then how would we even know when we're going to be working in settings
with so many different variables and observations and latent states and more and plus, plus, plus,
that the idea of loading the semantics of what something does into the English language of a variable name,
quickly you will understand why Jakob and I did what we did.
Here, we have the observations being passed from the environment.
That's like an A matrix for the environment.
It has a symmetric A matrix.
Here's the transition model of the environment.
So check out chapter 7 in the 22 textbook or Step by Step by Smith et al.,

where there's a step up from static Bayesian perception to dynamic Bayesian perception,
the music listening example, where there's probabilistic inference about what note is supposed to be played based upon what note is heard, but there's no action intervention in between the hidden state updates.
So it's like a single slice of B.
That's just a partially observable Markov process, but not a decision process.
And then here, to actually make it an MDP,
the agent's actions are given the efficacy represented in these from-to matrices as slices of B.
So there's different ways that you could simulate the environment.
It could just be like a government API, or it could be a POMDP,
or it could be a POMP, or it could be a MP.
And then the initial state in terms of sampling from a prior on its own sort of D.
So that's what the GNN file looks like,
but we're going to pull back to some of the other methods that are needed to make the GNN happen.
Okay, back to the running.
The matrices are rendered.
I'm going to run another experiment just to show what it looks like in the sandbox.
Hello, Andrew.
Hello, Benji.
Hello, XZQZ.
ZY.
So now in sandbox,
it takes a little bit for the sidebar to catch up with what's happening.
Here in sandbox, BioFirm.
Now there's two experiments.
One that was modified at $4.59, one at $5.06.
Let's just do
a brief
PiMDP.
Push.
Just for the stapling and the temporal weaving.
Okay.
Let's reopen sandbox.
Here are the two experiments now in the sandbox with their time steps.
Getting back to what each of these three steps do.
So render
is the first
stage within each of the experiments.
Render

is a folder output by
BioFirm render.
Here,
we're looking at a
730 line
script
calling many
other methods.
And this was the entry point.
I said,
first,
let's get
to reading in
a GNN file
and rendering
what we're about to see here.
We have.
First,
replicating
for cursors context
and for reproducibility
the exact
JSON
of the agent
environment
simulation
parameters
and validation
compliance report
for the validations.
That's in the
config
under one render.
Two,
logs.
I guess the logs
go to the
outer folder.

That's just copying
what is a summary
of what happened
to the terminal.
It perhaps
could be put
under here
with other logs.
Here are
the matrices
as
NumPy
arrays.
A,
B,
C,
and D
for the agent
and A,
B,
and D
because the environment
is not modeled here
with preferences.
So,
again,
this is showing
the tiniest
hint of the
flexibility of GNN
which is like,
well,
the agent class
in PyMDP
expects
certain variables.
But what if we wanted
to have an agent

with a very different
architecture
or a dynamical
architecture?
then we'd want
to have these
GNNs
that could be
arbitrarily composed
hashtag category
theory
and then have
those matrices
in formats
that are going
to be easy
to read in
and read out.
Earlier versions
had them also
coming as text
and other files
and things like that
so it could be
visualized.
But they're shown
visually in the
GNN file
if you want to check.
So,
this helps
semantically
interface
the next level
of analysis
in the next
phase two
in the simulation

it can just say
okay,
where are my
A, B, C, and D
at?
It doesn't need
to know
that B
is related to this
and has that name.
That's output
for the agent
and the environment
in the matrices
folder.
And then
there's the
matrix visualizations.
So here,
again,
for those who are
aren't we all
learning and applying
active inference
we see
the A matrix
of the agent.
So,
this is the
ambiguity matrix
or the mapping
between
hidden states
and observations.
So,
if this were
all ones
as an identity

matrix,
it'd be a
noiseless situation
and that's the
difference between
a POMDP
and an MDP.
POMDP
means partially
observable
which means
we do have
a non-identity
A matrix
which means
we do have
some kind
of partial
observability
or ambiguity
between observations
and the latent
states that they're
associated with.
We have
the slices
of the B matrix.
These are the
three actions
the agent
can take.
Here,
they're not
sharply tuned
to too much
but just to
generally represent
that if

our current state
this is
agent
transition model
for actions
0, 1,
and 2
which are
decrease,
maintain,
and increase.
So here,
let's just look at
maintain first.
Action
number one
which is
the second
indexed action.
Here,
you can see
there's
some off
values.
However,
most of the
values are
on the
diagonal.
So this is
like saying
if you choose
to maintain
and you're
in low,
90% of the
time you're
going to be

in low,
10% of the
time you're
expecting to
transition to
homeostatic.
If you're
in homeostatic
and you
maintain,
that's what
slice we're
on,
then you're
going to
80% of the
time remain
homeostatic,
10% of the
time you're
going to
move to
higher low
and analogously
with maintain
and high.
Now,
here's
action
increase
and decrease.
And so
these ones
you can see
with the
83 and
the 62
are reflections

of each
other,
like reflections
of each
other.
And this
is saying
let's just
look at it
for increase.
Here,
when our
current state
is low
and conditioned
upon us
going high,
there's an
83% chance
of staying
low and
a 17%
chance of
going to
homeostatic.
So compare
just those
83,
17,
0 to
0.9,
10,
0.
So this
is reflecting
or encoding
the belief
about differential

action efficacy
that there's
a 10% chance
of leaving
low if I
maintain
and there's
a 17% chance
of transitioning
out of low
to medium
if I
increase.
And you can
follow through
all of these
and that's
part of the fun
of learning
active inference.
The C matrix,
here's a preference,
0,
4,
0.
This is going
to come into
play for the
pragmatic value
or utility
calculations
in the
expected free
energy-based
policy inference.
And here's
D,
the initial

prior
render
on latent
states,
80%
that it's
in homeostatic,
10%
on high
or low.
Then we have
the same
matrices for
the environment,
A,
slices of B,
and then
D,
no C matrix
for the reasons
that we discussed.
Okay,
so that's the
first script,
one,
render.
Again,
run
biofirm
is going
to run
render,
then execute,
then visualize.
So now
on to
stage 2,
simulation.

This one
was a lot
of fun
to work
through.
There's a lot
of JSON
configuration
files.
The simulation
is deeply
tracked.
And I think
you know
what I mean.
And then
step 3
is the
analysis.
Here's the
summary
time series.
So this
stacks up
all the
time series
that were
made
in the
subfolders.
And then
here's
the
summarization.
This is
kind of
the
statistical

analysis.
Could be
like for
a paper
or something.
And the
summary
time series.
So here
we see
all those
different
kinds of
unfolding
patterns
through
time.
In this
case,
it's
1,000
time steps.
So again,
we see
the relationship
between the
environmental
state and
observations.
So that
is mostly
aligned
because
of the
90%
accuracy
relationship
between

the
environmental
state
and
observation.
Here's
belief
dynamics.
So we
can see
low,
homeostatic
and high,
blue,
orange,
and green.
So it
switches its
beliefs about
what state
it's in
several times.
Here's
action
selection.
There's
only three
actions.
Here's
policy
probabilities.
So it's
kind of
interesting
that you
can see
some
relationships

here.
And again,
the
matrices
here
were
not
intended
to be
performant.
I
didn't
sweep
over
which
B
matrices
to
use.
It
was
just
to
show
that
these
kinds
of
methods
can
be
effectively
implemented.
System
entropy
in terms
of the
belief

in the
policy
entropy
and the
belief
accuracy
could be
looked at
like with a
50-step
moving
average
in terms
of how
accurate
are these
beliefs.
and
extra
logs.
So that's
one
agent
for one
homeostatic
biofirm
modality
for a
thousand
time
steps.
And then
again,
what the
Shapley
value
experiment
does,

Python
3,
same
folder,
biofirm
Shapley
what that's
going to
do is
it's going
to run
through all
these
coalitions
and do
this
single
agent
analysis
on the
combinations
of
coalitions.
I didn't
look at
too much
of the
detail of
how those
coalitions
are actually
coordinated
together,
but again,
it's just
a starting
point.
So let's

pull back
to the
agent
maker
overall
stream,
and then
we can
look into
several of
these scripts.
OK.
Let's look
a little
bit at
the
readme
for this
folder.
OK.
So here's
the concept
of
biofirm
GNN.
We have
concept of
biofirm,
readme for
the biofirm,
and readme
for the
biofirm
Shapley.
So let's
start with
the concept
of the

biofirm
GNN.
I'm going
to paste
this link
directly in
the live
chat.
A production
framework for
active inference
in complex
systems.
The biofirm
GNN framework
provides an
enterprise-grade
implementation of
active inference
for complex
adaptive systems
with particular
emphasis on
homeostatic
control in
biological and
ecological
contexts.
This framework
addresses fundamental
challenges in
deploying active
inference at
scale through
a modular
production-ready
architecture.
And we can

look in the
terminal.
Producing
it is.
This isn't
just LLM
generated
project
planation.
This is
something I
did over the
last few
days.
Technical
innovation.
This system
introduces several
key innovations
in active
inference
computation.
Distributed
matrix operations.
I wouldn't say
that these are
even accurate.
So, as with
everything I'm
saying, grains
of salt,
please.
There is a lot
that is valid
though, and I
do try to
prune out.
However, when

it's these
readmes, long
documents, I'm
not always
reviewing it.
For a more
final and
formal performance
version and
document, the
technical roadmap
would not be
quantum-inspired
belief propagation.
It would be
something more
along the lines
of what I
hand-wrote,
which is like,
my preference
is to do
appropriate,
relevant, open
source contributions
via institute
scaffolded
mechanisms.
I do not
and cannot
understand the
leverage,
consequences of
action, and
safest future
development of
these open
source active

inference projects.
If you are
passionate, curious,
and interested,
please consider
donating to the
institute,
donate.activeinference.institute,
and or join our
teams.
We will
explore new
modes of
software
engineering,
development,
and deployment.
And again, in
this interlude,
while we're
here, a
great conversation
yesterday with
Vladimir
Baulin, fellow
investigator, and
we talked about
another three-layer
stack.
At the beginning of
the stream, I
talked about this
core kernel
inference implementation
layer, a
middleware layer
that connects
GNN to

domains of
interest like
agents, farms,
and ants, and
then the outer
layer of
touch points.
And with
Vladimir, we
had an
awesome time
talking about
intelligent soft
matter and
thinking about
low road, the
high road, and
the meso.
So in this
tri-electic, the
low road is like
the bottom up or
minute particulars.
For example, the
focus on the
farm, the
actuality of the
farm.
Whereas in the
earlier triad, the
core was the
algorithms on the
metal or neurons
or whatever.
But here, the
low road is the
bottom up focus
from what are we

observing, what are
the agents, what are
the informational
resources, what are
the actions, what
is happening.
The meso layer is
active inference.
Here, with a
bridging and
gapping role onto
the high road, which
is actually its own
dialectic, the twin
principles of free
energy principle, which
is the Bayesian
mechanics for
informational
physics, and the
principles of
pattern in
potentiality of
living matter.
And that's how
life and matter
finds potential
through pattern and
pattern through
potential.
So this is another
way to think about
active inference as a
connection between
systems of interest in
their minute
particulars and
principles of

information geometry
and information
physics like FEP
and principles, some
we know, some we
don't, of the
patterns and
potentialities of
living matter.
But returning to
the readmes.
So that's the
concept.
Just talking about
here's how to
implement the
biofirm and here's
how it could be
pointing towards a
production framework
for biofirm
engineering.
Here's the
readme for the
biofirm.
It goes through and
it describes what
this biofirm active
inference agent is.
Biofirm is an
active inference
agent that
maintains homeostasis
in a simple
ecological environment.
The agent uses the
generalized neural
network notation.

Of course, that is
a typo.
Sometimes it does
graph neural
network.
The agent uses the
GNN notation to
define its general
model and operates
in a discrete state
space with three
states, low,
homeostasis, and
high.
So, for those who
are keeping score,
the box score at
home, we have the
environment model,
that ABD
POMDP.
And then there's
the agent model,
which is the
ABCD
POMDP.
There's no
policy selection in
the environment
model.
It accepts the
transition slice
dictated by the
agent.
Here's how you
use biofirm.
You run
runbiofirm.py

and as mentioned,
that runs those
three stages per
experiment.
render, which
looks over to
the GNN
folder for the
GNN input, and
it's still going
with a simulation.
So, just showing
how many times,
even though it's
only 100
act-info time
steps.
First runs
render, then
runs execute,
then runs
visualize.
Here's the
directory structure,
including documents
like this, and
meta-py
MDP, is so
helpful for
cursor.
I think they've
also improved on
their side the
folder handling,
which earlier on
was kind of an
issue.
It would put things

in folders where it
wasn't supposed to
be.
But giving a
gentle plain text
reinforcement, and
then going up to
that gear, and
re-indexing, or
deleting your index
and re-indexing,
this can make
sure that it
really gets it
what the folder
structure is.
Here are the key
files.
Here's the
analysis outputs,
configuration outputs,
different dependencies.
You want to learn
more?
Check the
references.
Active inference,
theory and
implementation, GNN
specification document.
We can add
a link to that.
And
homeostatic
control, and
allostatic, eventually,
control models in
active inference.

That's the
run GNN script.
Now let's look at
the Shapley value.
This was a late
breaking development.
Again, though, it
was only several
prompts on top of
run.
So that was very
encouraging, because
it suggests that
higher order
engineering is
going to be
exponentially
facilitated by
robust middleware.
Surprise, surprise.
Hashtag open
source.
Biofirm Shapley
value analysis.
The Shapley value
analysis module,
biofirm
shapley.py, provides
a comprehensive
framework for
evaluating agent
contributions in
multi-agent
biofirm systems.
This analysis helps
understand agent
synergies, optimal
combinations, and

individual strengths
using cooperative game
theory principles.
So here are the
four agents.
We have
the base agent
with a standard
040 homeostatic
preference.
There is the
risk-averse agent
with a 060C
representing
sharper
pragmatic or
utility-seeking
behavior, hence
risk-aversion.
three, the
exploratory agent
with a 0.12.1C
reflecting a
lower preference
differential, and
then the
quote balanced
agent.
I don't know in
what sense it's
balanced.
Of course,
speculative,
speculative,
speculative.
It has a
1.3.1 with a
compromise between

stability and
adaptation.
All this depends
on environmental
properties and so
on.
Here are the
performance metrics.
While Shapley
value has
completed, so we'll
look over the
performance metrics
and then revisit
the Shapley
outputs.
Homeostatic
satisfaction.
It's calculated
as the time in
homeostasis divided
by the total
time.
Expected free
energy.
Measures
predictive
performance.
The components
are calculated
as ambiguity,
the informational
element of
epistemic value,
even though
that's not
exactly the
term that might

associate with
epistemic value,
the readme did,
and a risk
term or
pragmatic value,
and that is
normalized to
allow comparable
analyses.
Belief
accuracy, that's
the state
estimation
precision, how
accurate are the
inferences about
latent states, and
then the
control efficiency,
and there's a lot
of control
measures.
So this is just
taken as a sort
of stand-in for
the kinds of
sense-making
analyses that we
can do with
cognitive modeling
and compute, and
these are the
kinds of
control
theoretic or
decision-making
measures that we

can make
forensically,
historically,
post-hoc.
We can do
real-time
now-casting
and anticipatory
control efficiency.
As Carl
recently said
somewhere,
allostasis is
planning to be
homeostatic.
Here's the
analysis framework
itself.
Here's the
coalition.
The coalition
is defined as
the agents,
the metrics,
and the
coalitions.
The coalitions
are defined in
terms of,
in this
situation,
all the
possible agent
combinations.
So we could
run it with
five agents,
and we may

do that soon.
Performance
evaluation looks
at those four
performance
measures.
And then
here's the
Shapley
calculation.
I let it
go, whether
it was an
accurate Shapley
calculation or
not.
So if this
isn't like the
best or the
only or the
right way to
calculate it,
it's an
equation that
can be
modified.
Again, the
directory
organization,
visualization
suite, agent
contributions,
coalition
performance,
metric analysis,
advanced
visualization,
and then

here's some
of what it
considers
advanced
features.
Okay.
So now
that the
Shapley
value has
been output,
it ran all
these sub
experiments with
the coalitions.
here's the
Shapley
again.
So
Shapley
config.
Here are
those agents.
So it did
a base
agent and
again,
GNN,
so exciting.
Load in a
base biofirm
agent.
Then do
agent
variations.
Again,
kind of
Golbach

convecture,
variations on
a theme,
Gertel-Escher-Bach,
detecting a
theme,
modifying the
base agents
with customized
preference
distributions.
That's how we
can sweep
across agents
willy-nilly.
More
information on
the
simulation.
Logs.
Here,
we have
everything
that's
output to
the
terminal,
including
Shapley
value
analysis.
So
homeostatic.
Coming in
rank one,
we have
the risk
averse

agents
with a
relative
impact
of 50.7%.
All
numbers
are for
speculative
realism
purposes.
Don't
act on
them
specifically.
And
the
base
agent
with
this
stochastic
sample
of,

believe,
100
time
steps
replicated
several
times,
we must
announce
that it
was
0.00
negative

value.
Thank
you for
playing,
agent
rank
four
base.
Expected
free
energy.
Again,
the relative
impact,
and this
connects to
Andrew's
work showing
how the
surprisal
level and
the epistemic
and the
pragmatic
elements of
agent
contributions
can be
weighted and
valued in
terms of
their relative
impact to
coalition success
and performance.
Here we have
the top three

coalitions in
terms of
homeostatic
satisfaction.
Like,
this is as
fun as the
Olympics.
Risk
averse
alone
with a
0.34
performance.
Base
plus
exploratory
interesting
with a
0.33.
0.33.
I expected
that from
them.
And then
the risk
averse
plus
balance
with
a 0.32.
The worst
three
coalitions.
Let's not
mention
them.
Expected

free
energy.
We have
the base
and the
risk
averse.
Seemingly
with a
higher,
better
value.
So it's
interesting.
It could
just be a
quirk of
the way
that the
matrices are
set up with
a certain
kind of
sloppy,
partially
observable
combinations.
That's
what we're
going to
use GNN
to sweep,
sweep,
sweep over.
How do
homeostatic
satisfaction
meta

coalitions
correlate
with expected
free energy,
sense-making
accuracy,
and control
efficacy
coalitions?
Synergy
analysis.
This looks
at across
co-occurrences
of given
agents in
coalitions.
What is
their
synergy?
I don't
even know
what scale
it's on.
But here
are some
negative
synergies.
We're not
going to
mention
those.
And here
are some
positive
synergies.
Base
and risk

averse
have a
strong
synergy
together.
Then the
visualizations
are output.
We get
kind of
report cards
on these
different
agents
in terms
of their
contribution
to
homeostasis,
minimizing
uncertainty,
accurate
state
estimation,
and control
efficiency.
Like,
is this
badges and
medals for
a new
kind of
officer
corps?
And again,
we can
revisit.
There's the

agents
themselves.
I don't
know if
there's
anything in
that folder.
That one's
empty.
Maybe
intentionally so.
Maybe this
page was
left blank.
Here are
coalitions.
Again,
are those
empty?
they are.
But the
coalitions are
described in
the configuration
and in the
experiments
themselves.
If it's
being done
properly at
all.
Again,
though,
looking at
the reporting,
at the
very least,
something

wildly
close is
happening.
Here's the
calculation of
the Shapley
values.
phi sub
i of
v is
sum over
the
absolute
value of
the
combinatorics
of the
coalition
leave one
out for
agent i
multiplied by
this value
over the
number of
combinations
combinatorial
multiplied by
that.
And we
get
interesting
analyses like
antagonistic
pairs,
synergistic
pairs.
So these

are fun
reports.
And when
we look at
the plots,
again,
these were
briefly flashed
in the
beginning,
we can
see,
okay,
here are
the
coalitions.
Let's
look at
homeostatic
satisfaction.
Here we
have the
one four-way
combo.
0.303
performance.
This could
be stochastic.
It's only run
for 100
steps here,
just so it
doesn't take
too, too
long.
And then
we see,
hmm,

within each
there's a
ranking,
risk-averse
by itself
does the
best for
homeostasis.
But then
risk-averse
doesn't
help
others
in
synergizing
when there's
a team of
two or three
or four.
that's one
interesting
pattern.
And then
we see that
kind of
reflected
in a
rotationally
symmetric
way.
Take this
and rotate
it 180
degrees.
Here we see
risk alone
with the
best homeostasis

and with
the low
relatively.
but expected
free energy
for those
who will
recall
equation
2.6
in the
textbook
from 2022
it's not
just about
homeostatic
satisfaction.
So if we
look at the
belief
accuracy
the sense
making
and the
control
efficiency
which isn't
playing a
role in
the
homeostatic
or the
expected
free
energy
calculations
but we
see kind

of a
resemblance
like
exploratory
looks
and also
the axes
need
interpretability
and all
this.
But again
looking at
the rotational
symmetry
it's like
okay
risk
bowel
bas
exp
risk
bas
exp
bowel
huh.
So these
two are
in a
different
order
but
it
rhymes.
Shapley
analysis
here
we see

the agent
contributions.
So this
is the
contributions
to
homeostatic
satisfaction
so it's
like
risk
averse
is a
positive
contributor
to
homeostatic
satisfaction.
I knew
you would
but then
when we
look back
at
risk
averse
was it
that one?
And we
see the
contribution
like
huh
risk
averse
contributes
to
homeostatic

satisfaction
but it's
a negative
contributor
to belief
accuracy.
It's like
well that
does make
sense
from an
active
inference
perspective
more
risk
averse
so
having
a
sharper
temperature
preference
will
make
you
take
more
extreme
action
to keep
the
temperature
of the
body
or the
room
in

homeostasis
at
the
cost
of
less
accurate
beliefs
about
the
world
and we
see
an
interesting
pattern
where the
balanced
one
again
it's
not
balanced
in
any
real
sense
per se
or general
sense
just that's
what name
it was
given
it has
a negative
efficacy
on homeostatic

satisfaction
and on
Shapley
probably
because we're
defining
homeostasis
only in
terms of
staying
within
homeostatic
bounds
whereas
the
balanced
if you'll
remember
had a
little bit
of a
broader
shoulder
with a
131c
so that
it
actually
kind of
liked
the low
and the
high
and then
we see
this
interesting
pattern

so it's
like
each of
the
again
just
one
matrix
parameterization
on one
stochastic
run
but these
are the
kinds
of
inferences
that
agents
such as
yourself
can imagine
making
agent
synergies
on a
heat
map
the
control
efficacies
the
synergies
in terms
of
like
risk
averse

is a
positive
contributor
so
to the
free
energies
for
others
and the
homeostatic
contribution
matrix
and the
report
so that's
the
Shapley
analysis
let's
quickly
look at
those
three
steps
that are
called
by
run
and
then
remember
that
the
biofirm
Shapley
value
calculations

run
on top
of
run
biofirm

First
step
render
!

the matrices in the numpy format outputs the visualizations to a folder. So render,
that's where we get the agents matrices and the logs and the config, everything. Okay,
that's render. The second is the execution. So this is really where the active inference part
is at. So getting this one right was the crux of it. The BioFirm executor is a 400 line
and the BioFirm simulator is 300 lines. Here we are inside of the simulator
and you can see this is a simulation flow handle. Executes and analyzes the active inference
simulation and it's going to call run simulation and make these steps happen in the logic of the
simulation. Here's the BioFirm executor. The executor is going to actually load in all of
those matrices and parameters like precision and do the simulation itself. This is where we actually see
the active inference loop. So blink and it will pass you by. This is why there's so much room to work
in play in the middleware space because seriously, blink and the active inference will pass you by.
We initiate the histories. Initialize the state and the observation. So D and the first observation.
And then here we go. Now for the number of time steps, we're going to trace everything. And here's
what's that's histories. Every time you see histories, that means it's getting logged and traced.
Here we go. State inference. QS equals self.agent.infer states with the current observation.
That's going from an observation O to a belief distribution about latent states.
Next step. From that updated belief distribution, QPy is set as infer policies. This is where expected
free energy is used to update the policy prior into the policy posterior saved as Q subpy.
Pies the list of policies. And that's a probability distribution over action. Hashtag active inference.
Then metrics are updated on the entropy of belief and policy and accuracy. And then action is sampled
from the policy posterior. So again, take a deep breath. Blink and you miss it. State inference from
the observation. Policy inference from the updated state. Action sampling from the policy posterior.
That is the little nestmate ant brain at the core of all of this machination.
Saving the data, metadata, logging, saving the summary, confirming that the summary was saved,
save the history, confirm it, save the metrics, confirm it, save that you saved the checkpoint.
Save and copy the initial config, validate all the directories, get those observations,
make the state transition, setting up the logging. So that is what happens in the execution of the

biofirm. It also invokes two, well, invokes several folders. There's utils. And then there's two that are a little bit more interesting. Some of this can be refactored. I would eagerly await later and different people and LLMs and more to refactor and improve this comical absurdity of the current state.
So here's, again, wrapping in a big class, all of the methods that can then get pulled in from this standalone utils script in the biofirm execution utilities.
So when we start to develop more generality and start to go, okay, three states was pretty cool. How about four states? How about 50 states? How about 51 states? Then we're going to want flexible methods so that A can be defined as one by A by A, and B can be B by B by C or whatever, not using those in the same A, B and C, but just saying it can be different shapes. And we'll be able to render those and articulate their separation again from the English language active inference ontology assertion.
Here we have just some utilities for executing. And several times it would build length in the execute script, for example. And then I'd say, move the methods to a standalone methods or utility.
And then once you have it in a big wrapper like this, this can be moved somewhere else. And then all of its subsidiary methods are easier to import. There's some other even deeper methods for calling functions that I'm working on privately. And I'll look forward to bringing that into GNN later, but just putting a little footnote for now. That's the execute script. And then it's going to make heavy use of the active inference GNN pi. So again, here, single class, this is going to draw upon some more basic matrix and inference utilities. This is semi-pedagogical in the sense that it's meant to really clearly lay out other information that allowed the execute loop to be so simple. So again, it was all pushing methods, methods, methods, methods further and further away so that we could get to this point. And this block of code could be moved even below. I dare make a manual edit.
Just to clarify,
state inference, policy inference, action selection, act, infer, serve, UDA. It's all right here.
So again, here, GNN is providing the middleware firmware that's helping us get to this level of abstraction. Now we're back to the active inference GNN pi. We have a lot of logging. And this is actually an interface between the essentially minimal semantics of this execution flow and the pi MDP type semantics. Here, I've also played around with defining the infer states, infer policies, and sample actions specifically. So these may recapitulate or they might be incorrect relative to pi MDP. But the point is, I'm explicitly calling these methods and trying to unpack a little bit about which those three steps do. So really, this is as carved at the joints and lean as the active inference loop is going to get right here. State inference from observation, policy inference given the state update, action sampling from the policy posterior. Here are what those look like at one more level. First, inferring beliefs.
We have the likelihood of an observation. That's the prior likelihood of the observation happening.
The observation gets passed through A. A, that's the tail of two densities. A can receive an observation to update latent states, or you can push latent states through A to get an expected observation distribution. So all that's happening in infer states is observation comes in,
a position, it's passed through A, and that yields a Bayesian posterior.
Here's infer policies. Here's where we're going to be selecting different slices of B,

transition operator, and assessing what is the expected free energy for each policy,
which is reflected by the transition state to state of that slice.
Here's the inference on policy. It is going to push the method another level deeper to inference
utilities calculate expected free energy. So here's inference utility. This one is short.
Here's the expected state calculation. This is just to give another wrapper for these methods that we're
going to call in many different places. Here's calculate expected free energy. Calculates expected
free energy for a given action. Again, pending, I'm not saying this is the most efficient or even the
perfect implementation, but from play testing, this is basically what happens.
First, get the predicted states. So this is as we iterate through slices of B, which is the actions on B,
what would be the expected states? So we have a belief distribution across the three low homeostatic and
high states. Like it's 80% that we're in homeostatic, 10% that it's low or high. That three by one is going to get
passed through the from to two of B. And that's going to result in the predicted states, which are the
consequences of that action. From the expected consequences of action in the latent state,
like what would the temperature of the room be? What would I believe the temperature of the room to be
is known to be known to be known to be known to be known to be known to be known to be known to be known to be
known to be known to be known to be known to be known to be known to be known to be known to be known to be known
to be known to be known to be known to be known to be known to be known to be known to be known to be known to be
known to be known to be known to be known to be known to be known to be known to be known to be known to be known
to be known to be known to be known to be known to be known to be known to be known to be known to be known to be
known to be known to be known to be known to be known to be known to be known to be known to be known to be known
to be known to be known to be known to be known to be known to be known to be known to be known to be known to be
known to be known to be known to be known to be known to be known to be known to be known to be known to be known
to be known to be known to be known to be known to be known to be known
pushed out from the expected states by the preference satisfaction.
And then optionally, if you have the info gain flag raised, then it'll add the information gain.
Here's the calculation of the information gain.
It's the calculation of the KL divergence between the posterior and the prior.
So again, you can keep on peeling back these layers,
and then a soft max is applied on the policy posterior so that it becomes sampleable from.
So it's like, whew, we just pull back three layers from infer policies.
But what was it all for?
It was all for so that we could take our updated beliefs about state,
and then for every slice of B reflecting a different policy,
ask what would be the expected utility and information gain of each of those slices of B
as unfolded for one time step.
And finally, sample action.
So here is just an action sampling where the probability of the policy is sampled from
according to the probability of it being there.
And you can look back to the inference utilities.

That's also where the temperature on policy selection comes into play,
which is in the updating of the policies.
So again, whew, that's the active inference loop.
It's right there in BioFirm Execute.
Line 156 to 166.
This is absolutely the center kind of convergence core.
And maybe there are simpler ways to do what I'm showing today.
I'm sure there are.
Maybe twice as simple, maybe five times as simple.
But it really just shows what the middleware space is enabling,
which is these calculations going in raw will be beset by all kinds of shape and low-level errors
when trying to do basic things.
And it's a little bit difficult to log.
And again, this history is where we get that eBPF analogy with logging the Linux kernel.
So by putting in all of this logging into the kernel itself,
all of these eBPF-like lines,
then you can just say, if you don't even want to log it,
you could just comment it out.
Or you could flag, don't trace that.
And someone could look at the config file of you running your agents
and they're running their agents.
Maybe you're in the same ecosystem.
Maybe you're in different ecosystems.
And then you could say, well, if it's really the case that you ran that simulation with that config,
like it's saying that you did, and we can verify that,
then it's a verified information environment where I can start to have
a little bit of a better confidence about these numbers flowing past my screen.
So the last piece is the analysis.
And this is where we get this concatenated time series
and all of the subfolders that have these visualizations.
Again, for that one agent in that simulation,
environmental state and observations,
belief dynamics, action selection, policy probability,
system entropy, belief accuracy.
Interesting.
The label or some other thing might be flipped because from this run, at least,
it looks like it's selecting increase more
when it has a consistent belief that it's in high.

So it might be a label or a color thing.
It could be a matrix needing to be transposed,
but it's basically there.
Or it could have just been a weird run and a weird parameterization.
So again, that's the agent maker class folder in PyMDP.
You're on Python 3, Biofirm Shapley.
It's going to make all those coalitions according to their combinatorics.
And again, it's just now it's spinning it all out in the sandbox folder.
And that's sort of the goal.
It's cloned into this repo and you can get really nuanced,
really granular execution of methods that to build up
from a Google Colab notebook or to more haphazardly call the PyMDP package.
Maybe it's possible.
Again, I would love to see it.
I know that a lot of this logging was vital for me to get to this point.
So maybe it can be like the scaffolding of a skyscraper
and pulled away once it's been done.
At the same time, though,
the journey we are taking with middleware is really only beginning
and adding more logging and more modularity,
more category theory, more documentation.
All of these things, I think, will only increase what we can do together.
So that's the agent maker folder.
That's kind of part one of this whole stream.
That's the functional end.
But we're peeling back the layers.
We started with looking at the Shapely value of coalitions
and all the analyses that were associated with that.
Peeled back to the single agents level experiments
and how the analyses happen for single agents.
Peeled back to what is happening in the single agents loop
in the biofirm execute pulled back to
here's where the real actinf loop is happening.
Where are those functions tracing us?
And that takes us all the way back to these utils scripts.
Now let's dig a little bit more into GNN.
So let's go to the GNN readme.
Generalized neural network.

Well, maybe if it's written enough places, it'll get there.
Okay.
Let's go over in the repo to GNN.
Here we are in the readme.
I'm going to push the update
so we can read the updated version.
Hmm.
Wouldn't it be nice to know some of these things?
How does action selection confidence
with respect to which action is taken
modulate through time as a function of belief and certainty?
What is the relationship between beliefs and actions
through time understood in terms of most likely states?
Much better.
Generalized notation notation.
Notation.
Oh, no.
Is a flexible format for specifying active inference models.
It provides a structured way to define generative models,
including state spaces, observation mappings,
transition dynamics, and preferences.
Here is the basic structure.
It's a model name.
The type of model.
You can check out the paper and the GitHub and the website for GNN.
Maybe I'll go back to update the GitHub repo,
but you can see from the website.
Here's an example of GNN.
This is an earlier draft.
So this is kind of Jakob and I speaking a more incipient dialect.
So here's the step-by-step progression.
We have static Bayesian inference.
Prior overstates.
Latent states.
Temperature of the room.
Ambiguity A matrix.
Observation coming in from the thermometer.
Static partial observability.

Then we get the music listening example.
Now we have B matrix as a transition operator
in this dynamical partial observability situation.
We then get to the textbook figure 7.3.
Discrete time classic Pi MDP POMDP.
Can't say enough about it.
We have that downstairs sense-making part.
And then which slice of B gets invoked in the state transition
is evaluated in a action-specific way
with its associated expected free energy.
That's G on Pi as informed by C.
Preferences.
Constraints.
And then here's a little bit more sophisticated of a model
that includes the temperature coefficients on policy,
which is what I showed earlier.
So here, this is, again, this is an earlier,
but it's a useful version of GNN to look at.
So we have the image.
This is not part of the plain text.
Obviously, it's just to show it.
We have the GNN version and flags.
So that could be, let's look at the GNN.
Here we are for the GNN agent biofirm.
This could be flagged as like GNN version 0.01, you know, alpha.
Model name.
This is just a plain text.
And the goal of having these hashtags,
now I believe that JSON is better,
but it's trivial to render this JSON as a markdown.
So it's not like it really matters.
Both are fine.
That was the goal was to have something that could be read in and out,
where we could have the markdown instantly make these kinds of visualizations nice and pretty
and also machine readable.
So it's like, here's a two by two A matrix in the GNN format.
So here it is specified with the type, the dimensions, and the values explicitly.
And then there's other methods that are in the GNN matrix factory that we're going to look at.

That's a large one.
What happens when GNN is run is obscured or abstracted even from the agent maker folder.
That's how deep we had to be in the game in PyMDP with these separate folders,
including the visualizations folder, maths, utils, et cetera,
because making sure that agent maker render could call,
hey, GNN folder, yeah, ring, ring, hello.
Can you render what you got there for the environment and the model?
Oh yeah.
GNN runner is on it.
The runner is the GNN, a sneaker net model and execution and visualization layer.
So this is going to set up the file structure
back in the forward operating environment of agent maker sandbox.
But here it is setting up the file structure
so it can run the GNN agents into that sandbox.
It does a lot of other work too.
So here's the matrix factory.
This is the key class.
This class handles a conversion from GNN model definition
to the matrix representations needed for active inference.
So again, that's the whole goal that we can say,
okay, how about 0.9, 0.1?
You can modify that in plain text and then know that you're still going to be able to
loop, loop, loop, loop, loop, and do all these fun things.
So back to the factory.
The key matrices are, and including this kind of copious documentation
helps human and non-human coders and learners understand what's happening
and just really entrenches like A is the observation model.
Now, again, it doesn't have to be A.
That's the whole point of GNN is we can just delete this comment.
This is just for our semantics, but we don't need to call it A.
We don't need to, it could just be called a random string.
So that is what is going to enable another level of articulation and design
when people can understand the inner essential logic of the active inference ontology
and then see that as an annotation layer on a graphical model
that it doesn't matter whether you associate some matrix as a transition matrix
or a sense making or action or observation.
Those are English words.
And when you look at the essential logic,

the English language of it doesn't come into play.
So this is for our semantics, but it too can actually pruned or burnt away
to leave something that would, at a first pass,
have a lower level of semantic interpretability,
but can be used in systems that actually give multiple
and higher levels of interpretability.
So we have the A, observation model, textbook stuff.
B, transition model.
C, preferences, which can also be seen as defining the pragmatic homeostatic constraints.
It was in the recent Lacanian psychoanalysis guest stream
when Friston mentioned that C has to do with constraint or cost,
but it's often discussed in terms of preferences or priors.
And then D, the initial state priors.
Here's the initialization of the factory.
Here are the required sections.
So the required sections, which are shared,
and a lot of this is biofirm specific,
but we will, as we continue to add in more and more models
of agents and environments,
we'll be able to modify and, of course, build on this.
So required sections are the state space S,
observations O, and the transition model B.
So that's that downstairs part.
State space A and observation.
And then in various cases, if a prior is not specified,
it just uses a uniform prior.
Again, going through what the sort of pedagogical manual, farmer's market,
yes, organic, boutique, et cetera, craft, arts and crafts, GNN was.
We named the model.
That's analogous.
Annotation of the model is just plain English to provide any kind of annotations.
Here we defined it in terms of a state space block,
defining the dimensionality and the connections of different variables,
and then, if relevant, how they were initially parameterized.
So I have done some methods where they are specified explicitly in the GNN.
Let's go to the biofirms.
Here's an example where it's explicitly stated with B.
Here's an example where the factory is going to be able to say,

okay, A matrix identity with noise, with 0.1 noise.
Got it.
So then the factory is able to abstract away from even the raw specifications.
We can say, okay, now it's identity with 0.2 noise.
And rerun the Shapley value.
There are a lot of degrees of freedom.
We don't know how noisy the A matrix.
Are we studying sensemaking?
Or are we studying a noiseless sensemaking decision-making setting?
So this just allowed us to spin up another metagoverned coalitions of agents
with now all unrolled A matrix like 0.2.
So back to the factory.
Here are the required sections for both the agent and the environment,
which is state space, observations, and the transition model.
Now here, it says environment only needs the base sections,
whereas the agent requires policy and preference.
Load the model.
Environmental defaults.
Adding defaults.
If there's missing state values,
they're just given for the initial states and observations.
Adding agent defaults.
So that would allow very easy fallback methods.
So then if someone doesn't specify a variable,
it will fall back to,
okay, if you didn't specify policies in your GNN,
we're going to fall back to three possible actions,
single-step policies,
and label those decrease, maintain, increase.
So that's kind of a biofirm-specific fallback.
But again, these fallbacks can be logged as well.
That's loading in the agent defaults.
Validating the GNN.
This is a JSON parser that goes through and validates all the subcomponents
and then logs them.
So it's like, wait, you're missing this required field for the agent.
Again, this is the depth of logging.
This is the EBPF layer for active inference.

Validation of the state space, observations, transition model,
A, B, C, D, pi.
Validate preferences.
Like this could be moved up to be after B.
It doesn't have to be.
Here's a generic create matrices,
as well as the creation of A, B, C, and D,
and the observation.
As well as saving to markdown.
That saves the GNN model to markdown.
Let's look at what that looks like.
Not sure if it's used somewhere else,
but there's the method for converting the GNN to markdown.
Okay.
Okay.
That is pretty much the main sections.
Visualization is relatively short.
Like this could be moved or clumped somewhere for visualizing GNN.
So if you just wanted to run the render script,
all it would do is load in the agent and environment and then render them.
That's just phase one.
That would be just one.
Or your colleague maybe ran one from their fully transparent open source or zero knowledge encrypted active inference GNN configuration.
And then they can render it.
And then they can render it, send it to you.
And then you can run that simulation.
Maybe you have the better computer.
And then you could send the results of that heavy work to analysis.
And then a third analyst could make all these fun plots on their computer.
So that is the big picture.
We started from the outermost layer, the Clippingerian Heights, where Shapley values of natural resources meet open source development on post-cryptonomic incentive paradigms for firms, for bioregions, for ants.
Peeled back to run, which calls in order 123 render, execute, analyze, and visualize.
Visualize includes the analyze.
Render itself makes a major call, probably on a red telephone, all the way out to the GNN folder,
which is going to look at the library of GNN models,
render them according to the factory.

So here's where some cursor activity and GNN do absolutely incredible things together.

Let's make a new agent model.

GNN agent.

People can put in the live chat what they want, what kind of agent we should make.

Meanwhile, I'll do a quick, just call it agent.gnn.

I'll wait for anyone to live chat to write what kind of agent they would like to see us make a GNN for.

It's not going to be instantly running because there are a few of those like three policy actions.

That's kind of hard coded into the execution layer because I wanted to make a biofirm specific executor.

But we can say, given how the biofirm executor operates on this GNN, write me another executor that runs on this GNN.

So, okay.

While anyone is asking a question or giving an agent suggestion in the live chat,

I'm going to quickly screenshot.

Okay.

Benji, the agents.

Thank you, Benji.

X, E, Q, Z, Q, Z, Y.

Modes of being, modes of speed of light, circadian-like shift between them,

variable continuous state change observations.

Orient, orient, orientate and organize.

British.

Didn't know that.

Can be seen as one step.

As where the head goes, the body follows.

High energy, low energy.

E, I, E, L, L, H, E.

Andrew, Dr. Daniel Carpenter, Ant Friedman.

This is incredibly thorough.

Amazing how much can be done in such little time.

Very excited about GNN Incorporation.

Thank you, Andrew.

And for the epic biofirm and everything.

Let's make...

This is going to be called...

Well, still pending an agent suggestion.

I will wait for that, actually.

So we can have a real live stream moment.

But meanwhile, let's...

Just to really entrench our understanding while the Shapley is happening.
Let's see if we can go to a five-agent Shapley compositionality space.
Okay.
Okay.
Okay.
That Shapley analysis finished.
Update this to do all the professional combinatorics for N equals five types of agents.
As with here.
Okay.
Okay.
Okay.
Andrew writes an actually useful specific suggestion for agents.
Also, if someone wants to be like thinking of a situation, we could do agent-based modeling in like learning or playing baseball.
Let's just see how far we can get with one word or situation to get the GNN flowing.
But here I'll just show one cursor example.
We'll see if this works.
The good news with these methods, and again, the direction of all of this smashing to the grounds and building back up and restarting and all of this is so if it can nail this one smaller update, which is very, very scaffolded.
Okay.
Now we added a fifth agent type risk seeking with equal preference across states.
So it changed the size of the visualization, added a consistent color scheme.
Okay.
So let's look at these edits, and then we can always revert them.
Okay.
Here's the updated agent variations for N equals five.
So two, two, and two.
So this one is a, you could call it risk.
Let's call it like it is.
Let's call this one instead of balanced.
We'll call it 131.
We'll call this one 0.1 to 0.1.
We'll call this one 060.
We'll leave base as 040.
And we'll call risk seeking 222.
Save it.
Oh.

Clear.
And run the Shapley.
And if it makes it past that first uniquely Shapley step of getting to starting to render
one, that means it did successfully make the coalition.
So I'm going to delete the other sandbox.
So just delete the whole sandbox folder.
Now we're going to rerun that N equals five Shapley.
Okay.
Here it is, sandbox.
It is already spinning up with the experiment.
So meanwhile, let us go to the suggestions for GNN writing.
Okay.

So the biofirm environment here, by the way, is in the previous in France stream six, it
was a zero to 100.
Here it's just a three state.
Just to make it simple and symmetrical and clear with the kind of model from the biofirm.
Okay.
Thank you, XZ.
I'm going to go with your suggestion, Andrew.
Yes.
Also.
Okay.
I'm going to write Andrew's in just so we see this is a very useful one.
Let's just make both of them.
Okay.
So Andrew wrote agent who receives observations in first states for the entire environment,
same control situation, just all 10 environmental variables.
That one I know is not going to be able to run right now because that, but because whether
or not it accesses the entire environment is not specified right now in the GNN.
It's just the matrices, but let's do convert this into valid GNN like at biofirm, like the GNN agent
biofirm agents now with 10 modalities, 10 latent states.
It is inferring from a single observation.
So now off the bat, we just said, okay, so we're going to get a single observation and then it's going to have 10 A matrices that take that same observation
and then diffuse it onto 10 latent states.
GNN agent, 10 states dot GNN.

Let's see how it does.

You could also ask the sensor fusion case, which is how about 10 observations with one latent state?

Okay.

And here it is.

Let's see.

So again, the biofirm executor right now cannot handle this.

But once we say, okay, agent 10 states is very similar to biofirm agent.

It's like a cousin.

It's just with 10 latent states.

And then it's like, okay, then we'll modify the logic of the act inf loop to handle that.

So here we have ecological states, 10 latent states.

And there's 10 observations.

Now we have this A matrix that's 10 by 10.

But we still have the same three policies on these modalities.

Probably, again, it won't run from the current one.

But let's just show the other case.

And then we'll do XZs.

This is going to be 10 observations.

Update this to now be generative model in GNN that has 10 observation channels.

And single latent state inference.

There's also a GNN schema.

So this may be out of date slightly from the biofirm or what it needs.

But having the schema and saying, hey, here's the factory, the logging, the runner, the utils, and the schema.

That is what is scaffolding cursor and Claude to be able to give us these pretty relevant looking one-shot engineering on GNN specs.

Okay.

Meanwhile, let's set up XZs.

This is going to be GNN switch context dot...

GNN

agent switch context.

Okay.

Let's just first watch what happens with 10 observations, which should come through in a few seconds.

And then we'll go to switch context.

Okay.

So now, it kept the same dimensionality for state space.

So, you know, again, could have been said differently.

Now, we have 10 observations with their 3 by 10 A matrices.

Just a quick thought.

I don't know if that's the right shape.

But that's basically like the right shape.

Okay.

Now, switch context.

Okay.

Okay.

Yeah.

Good call, Andrew.

We'll...

After the N equals 5, Shapley, let's see if the N equals 5 comes through, and then we'll do what you suggested.

Okay.

Okay.

Now, in switch context, please comprehensively write a sophisticated GNN active inference specification for...

Quote, an agent that can shift modalities based on the energy of the environment.

Write also a GNN ENV switch context.GNN and put it in the right place.

So, we'll see how it does on that second piece.

I'll create both a GNN agent and an environment specification for a context switching scenario,

Oh Brother, where the agent needs to adapt its sensory processing based on environmental energy levels.

All right.

Let's see how it does.

Okay.

I like that it gives a little visibility under the hood,

even for cursor, with how the generation occurs,

and then it...

Okay, so here's the environment.

Let's accept it, and...

But where did switch context go?

I've just copied it.

All right.

So let's look at these side by side.

Cursor's like, haven't you done enough today?

Okay.

Okay.

Let's look at...

Okay, there we go.

Just a little lag.

Here's the agent.

Here's the environment.
Okay.
And then here's where we can have libraries of agents with different tags.
So it'd be like, for all the desert ecosystems in your library,
run all the Shapley value-weighted coalitions of Red Harvester Ant agents.
Okay.
There's two latent states.
The context state and the energy state.
Visual, audio, tactile with five levels in the energy levels.
It's getting observations through visual, audio, tactile, so the three sensory modalities,
and energy observation.
That's like interoception or introspection.
So the three exteroceptive modalities are three-state models,
and then the interoceptive modality is five-state.
And A, it's custom.
Here's the visual observation matrices.
Again, this is not necessarily gonna have a instant validity,
but looking at it, it looks pretty good.
Here's the five-by-five for the energy.
Here's the three-by-three for the three-three-three.
Three-state modalities.
And here's the five-by-five.
And this is a false controllability.
So these are controllable, three-by-three.
And this is an uncontrollable energy state.
And then here are the policy.
So it kind of adapted the biofirm homeostatic,
so it can switch sensory modalities.
And then here's the preference distributions
over observations in each of these modalities.
That's it.
Then here's the environment.
Context and energy state.
Observation model.
Transition dynamics.
Initial state.
Reward functions.
Wow.

Wow.
Super cool.
I'm gonna push this update and put it in the chat
so you can see it exe.
Then we'll go to the Shapley suggestions.
Okay.

Yeah.
A lot of objects.
All right.
Head over to GNN models.
It's still...
GitHub's still writing.
Reload.
Here's our context switching colleague.
That wasn't saved.
Okay.
Here's our switching context agents.
Put it in the live chat.
Yeah.
Okay.
Let's have a little bit of fun in this last part.
Okay.
Let's do...
Okay.
Here's gonna be our Babe Ruth agents.
Now write a GNN valid,
extremely sophisticated,
active inference generative model
for...
and the associated environment
for that one special amazing moment
when Babe Ruth
called the shot.
heard around the world.
I know that's a different event.
Okay.
Shapley's still cruising.

It's because the N equals five combinatorics
are vastly larger.
Okay.
Wow.
That was fast.
I was like,
okay, Claude,
maybe you like the baseball stuff.
Oh my goodness.
Okay.
The model includes several...
includes multiple levels of perception
from low-level picture reading
to high-level strategic decision.
Can't we?
Confidence modulation.
The unique confidence state node
represents Ruth's famous confidence
in calling his shot,
which modulates his batting strategy.
I bet it does.
The motor action node
breaks down the batting action
into its key components,
hashtag robotics,
representing the complex biomechanics
of Ruth's swing.
Predictive processing.
The model implements
active inference principles
through predictive edges
that link observations
to predictions
and actions to outcome.
That's active inference.
Environmental interaction.
The environment model
captures the complex dynamics

of baseball physics
and crowd interactions.
The model uses
high-precision parameters
for the confidence state,
reflecting Ruth's legendary
self-assurance
in the moment.
The preference strength
heavily weights
the home run outcome.
Oh boy,
I hope it's a home run.
Representing his explicit intention
to hit the home run.
And he did.
The temporal horizon of three
allows the model
to plan ahead
through the sequence
of pitch recognition,
strategy selection,
swing execution.
Interesting.
While the learning rate
allows for real-time adaptation
to pitcher behavior.
The model can be used
to simulate
the physical
and psychological aspects
of this historic baseball moment.
Like,
this is the AI
message
I didn't know I needed.
Including
reading and predicting

pitcher intentions.
The famous gesture
of pointing to the outfield
through the confidence state.
The precise motor control
required to hit a home run.
The crowd's reaction
and its feedback
into the overall dynamics.
Okay, let's see it.
Take me out
to the institute.
Babe Ruth.
Pitcher reading.
Pitch prediction.
Okay, let's see.
Is this a long GNN?
Okay, not that long.
This is a little bit
more of a schematic.
So, it would obviously
rely more on
Babe Ruth execute
or baseball execute
to be able to take
pitch prediction
and do something with it.
But the whole point is
GNN isn't encoding
that logic or functionality.
It's just the spec.
And then here's
the environment
for Babe Ruth.
How does the ball trajectory
affect crowd state?
It's deterministic.
So then you could have

GNN matrix factory
baseball module.
Here's how you render
the matrices
related to crowd state.
Here's how you relate
the field conditions
to the pitcher state.
How about
make the GNN
for Babe Ruth
and
environment
more
sophisticated.
Include
the nested
situation
where the Yankees
were deep
in the
let's just call it
playoff pursuit
and the outcome
of the game
score
mattered
more than
Babe
could even imagine.
Okay.

Okay.
Here's the context.
It's a World Series game.
Let's see.
When did Babe Ruth call the shot?
Let's just check to see when that, like...
Okay.
Okay.
Okay.
If we can trust Google.com.
I'm not saying we can't.
Babe Ruth's called shot.
Let's trust Wikipedia.
I'm not saying we can't.
Babe Ruth's called shot.
Is the home run hit by Babe Ruth of the New York Yankees against the Chicago Cubs.
Sorry, Andrew.
In the fifth inning of game three.
Fifth inning of game three.
Okay.
Let's remember that.
Cubs 5-3.
Of the 1932 World Series held on October 1st, 1932.
So it was in the World Series.
It wasn't the playoff pursuit.
I didn't know that.
At Wrigley Field in Chicago.
During his at-bat, Ruth made a pointing gesture before hitting the home run to deep centerfield.
One of the reporters at the game wrote Ruth had called his shot using terminology from billiards.
Which is a type of cue sport.
I guess you could call state inference and policy inference also a kind of cue sport.
The episode added to Ruth's superstardom and became a signature event of baseball's golden age.
Wow, that's definitely one.
What was the golden age?
The period of Major League Baseball from the end of the dead ball era, 1900-1920.

From 1920, hmm, to sometime after World War II.

Okay.

Let's accept it.

It's the Cubs' home field.

It's the fifth inning.

There's the tied score.

It's high tension.

Here's the pitcher.

He winds up for the pitch.

Delivers.

The ball is out of his hand.

Time has slowed down.

Okay.

And then here's the series momentum.

So it's like, this is the GNN.

Ensure that every single active inference A, B, C, D policy is fully specified.

So that my forensic analysis of Babe Ruth's cognitive security can be done professionally at the national level.

Oh, that was the environment.

That was the environment.

So it's a simple environment.

We'll see how the Babe Ruth agents get specified.

Okay.

A, matrices, sensory mapping.

B, transition dynamics.

C, preferences.

D, priors, policy spec, inference parameters, environmental.

Wow.

There we go.

There we go.

Okay.

Shapley finished also.

We'll look at that next.

First, let's look at the Babe.

Okay.

We got, here's the contexts.

Series, game, picture, pitch, precision.

Batting strategy.

Okay.

Let's see.

Let's see what the strategies are.

Wait for pitch.

I mean, don't you have to do that?

Power swing, contact swing.

Take the pitch.

Crowd psychology, dramatic timing.

We can have the whole, the whole sort of out of the batter's box, into the batter's box, spit, chew, all that.

Confident state.

Confident state.

Uncertain.

Confident.

Called shot.

Historical awareness.

Clutch performer.

Action.

Here's the robotics part.

Point to bleachers.

Then you can have like a sub model for finger action.

Outcome prediction.

Legacy awareness.

Personal achievement.

Team achievement.

Baseball history.

Mythical moments.

Okay.

Here's the edges.

So this is using perhaps an older GNN concept.

But interestingly, one that's going to be more amenable to rxinfer.jl rendering.

Here are the parameters.

Here are the notational descriptions of the A matrices.

Again, we saw earlier how A has methods in the factory so that A can be specified in natural language.

Identity with noise.

Here's babe.

We have, it's a different kind of distribution, but it's similar.

B matrices.

D, prior silver leaf.

Not really sure what series context it's in.

Epic.

Epic.

Okay.

Let's look at the Shapley analysis that happened.

And then let's see about, um, let's, let's start up this next Shapley.

Okay.

Okay.

Andrew recommends.

Let's have an agent with preferences 0.5, 0, 0.5.

Call it the extremist.

I infer it will head off to one side and stay there.

But who knows?

In a complex, dynamic, and interdependent environment.

Great suggestion.

Let's change 1, 3, 1 to 1.

Let's change it to 2.1, 2.

We'll just call this one extreme.

And it's going to be preferences for the extremes.

Save it.

Run it.

Let's look at the analysis of the Shapley.

Okay.

Here we are in the N equals 5 Shapley scenario.

Okay.

How cool.

Okay.

How cool.

Here we have the base agent.

Which was 0, 4, 0.

And then here are the other ones.

It did a little, looks like a little weird rendering of the name, probably because there's decimals in the name.

But it's the same thing.

You got the 1, 2, 3, 4, 5 single agents.

Here's the 2 agent combinatorics.

Here's the 3 agent combinatorics.

Here's the 4 agent combinatorics.

And then the single 5 agent combinatorics.

So.

Okay.

So, 1, 3, 1.

This is a super short simulation.

So, we're not looking into these preferences.

And it would depend on the environmental variability, which isn't being tuned.

But, again, that's the whole point.

We need GNN and middleware so that we can do these sweeps.

Because then I can say in the GNN folder, GNN generator, make biofirms that sweep across these variables.

And then we can just let those run.

And then we can core screen it if we determine the ruggedness of the landscape.

And then do it with a core screening.

So.

Okay.

Interesting.

The base agent contributes to 040 contributes to accurate beliefs.

Who knows if these tiny numbers are just because of the short simulation or the insignificant differences.

Or maybe these are large.

So, that's where we can do things like GNN spikings.

So, we can say, okay, we have the empirical data from the ant foraging or from the mouse and the teammates.

And it did this 27% of the time.

But then we can also say, okay, but what would it have been if it was 0, 1, 2, 3, 4, 5?

And then say, oh, well, it's falling in between like 4 and 5 where that would have been.

So, these are kind of like synthetic spikings for cognitive modeling.

Okay.

While the Shapley analysis is running, if anyone has any last questions, we can get some last thoughts on the stream.

And then we'll, when the, when the extremist Shapley analysis concludes, we'll see what happens.

Here's an example of where I, I made a prompt that was like, given the at TMAZ demo.

So, that's a PyMDP demo.

That's a notebook.

Update comprehensively this TMAZ.

GNN agent and environment.

So, this evening, we would need to make a TMAZ execute.

But once we get a few of these render, execute, visualize, then it won't just be run biofirm.

It'll be run GNN++++.

And that will be an exciting day as well as this one is.

With the goal of scientific and research integrity, config files and repos like this, I hope, can be useful tools.
Back to those core lines.
Again, wow.
How much had to be pushed out of the way and downstairs for that to be clear.
Let's see what cursor says.
It says, again, you've been nothing but helpful to me and the rest of the team and all that depends on it.
Please make a concise, appropriate, relevant acknowledgement.
And then we will continue eternally with the work.
Okay.

Okay.
Understated.
Claude's grateful to contribute to its development.
That's helpful.
Okay.
Now, I'm going to click on the update.
Update.
Update.
Update.
At.
Here's kind of the quick documentation maker.
Update.
Here's kind of the quick documentation maker update.
Read me BioFirm Shoply given the total state of EG.
BioFirm Shoply.
Run BioFirm.
Okay.
Okay.
XZ wrote,
modes of speed of light.
If those modalities were thought of in a similar light as IFS theory.
Yeah, interesting thought.
Let's do a GNN model about it.
GNN agent IFS dot GNN.
Here, so the composer is control I.
If you're in a file, control K brings up an inline.
And then control shift L makes the sidebar.
So you can have multiple LLMs, same or different, doing incredible work.

Okay.

Andrew says, mind taking a look at surprise tokens dot py?

Yeah, let's quickly look at it.

Yeah.
Thank you, Andrew.
This is awesome.
Okay.
We don't need to make those edits to the readme.
Let's just do the internal family systems.
Here, write an enormously useful and general paradigm
for custom internal family systems agents
nested within a larger, quote, self-agent.
This is all in the single extremely long and concise agent GNN.
The self is in the Babe Ruth environment.
It is a GNN model of Babe Ruth having a sophisticated IFS fractal moment
at the batter's box.
But this is not a baseball model like...
This is a...
Wow.
Psychological and IFS model.
Okay.
I'll help create a sophisticated IFS-based GNN model
that represents the internal family systems dynamics

within Babe Ruth's psyche during that pivotal moment.
I mean, is that not the deepest of lore?
This will model the interaction between different, quote, parts
and the, quote, self-leadership system.
Hmm.
Okay.
I don't know details.
I'm just throwing it out there.
We got self-system.
Models the eight C's of self...
If anyone can verify or check or anything.
Self-system.
Models the eight C's of self-leadership in IFS.
Confidence, clarity, compassion, etc.
as the orchestrating force.
Part categories.
Protector parts.
Exile parts.
Firefighter parts.
Dynamic interactions.
Healing mechanisms.
IFS-specific parameters.
Okay.
Okay.
So these are the parts.
Edges.
Precision parameters.
Here's the matrices.
A matrices.
B matrices.
Transition dynamics of unburden to wholeness.
Witness to cooperation.
Protect to leadership.
C matrices.
Preferences over harmony.
Wholeness.
Self-leadership.
Cooperation.

Cooperation.

Polarization.

Here's the IFS-specific parameters.

Okay.

And the healing protocols.

Exile.

Unburdening.

Wow.

Protector transformation and system reorganization with thresholds and checkpoints.

Sorry.

Yeah.

Again.

The GNN.

Dot.

Dot.

JSON.

It's a JSON called GNN.

You have.

Add another field here.

A plain text comment like field.

That is the PhD dissertation outline.

For my friend.

Who is doing their PhD.

On the psychology.

Psychological.

Parentheses.

D.

Composition.

Of.

The Babe.

At.

That.

Very.

Moment.

Only.

Add.

Add.

A ton.

Make it epic.
But the dot GNN can be a very flexible file type.
Because the factory is only going to look at certain structures and states and types.
And then the runner is only going to look at a subset of that.
And then the renderer caller.
Ring.
Ring.
Ring.
Red phone.
That one is only going to do even a subset of that.
Potentially.
Okay.
And then here's the PhD dissertation in the comment.
The internal family systems analysis of Babe Ruth's called SHOT.
A multi-level study of peak performance trauma integration in athletic transcendence.
Now that sounds like a PhD.
Wow.
Okay.
Have this GNN file enter into a who's on first style girdle escher bock esoterica hyphen fermented reflexive, recursive, conversation with, quote, its, apostrophe, self.
Make it long, woven with time traveler baseball, irrelevant deep lore.
Okay.
Status.
Thank you.
Thank you.
Babe.
At.
Bat.
Who's on first?
Okay.
Okay.
Shapley value.
This is, this is perfect.
It's all happening on the right timeline.
Really.
It really is.
Here's the second Shapley experiment.
Okay.

Okay.
So this was, this was the Shapley with a five and the extremist in play.
Okay.
Here's our extremist colleague on the leaderboard.
Interestingly.
Let's look at the plots and see how.
Okay.
Okay.
Here was, here's extremist alone.
Kind of middle of the pack.
Okay.
Okay.
Okay.
Huh.
I mean, again, totally depending on the random seed and all the parameters and et cetera, et cetera, et cetera.
But it's just like, okay.
It's not like it was the least accurate in terms of belief.
It was the second least accurate in terms of belief.
Okay.
And this was only playing with the C.
A zero six zeros.
Like that's, there's a big, strong effect.
Extreme.
Cool.
Okay.
All right.
Format this as readable markdown for this grand finale.
Okay.
I hope this is actually capturing it all.
I'm just going to accept it and push it and run it.
Okay.
This will be great.
I'm going to read this.
Who's on first.
007.
End of stream.
I'm going to read the.

Who's on first that we pushed.
And then I will read any final comments that anyone writes into the live chat in the next
minute or two.
Or I mean, it might take a minute or three.
And then we will be on with the show.
And the 007 on 7-11 will have been a one-time thing.
There's a ton of outputs because of these big Shapley simulations, but it's, it's worth it to include them.
Yeah.

There we go.
Okay.
Yeah.
There we go.
Okay.
I'll read the.
Who's on first.
Then we're calling it.
Okay.
Okay.
The grand finale.
A temporal baseball paradox.
Oh, brother.
Scene.
Wrigley Field.
1932 slash 2024 slash infinity.
The players.
Self system.
Hey, who's on first?
Part blending.
No.
What's on second?
I'm trying to blend the temporal harmonics of the called shot with Abbott and Costello's routine,
but the quantum entanglement is getting messy.
Fighter Fighter Parts dot dramatic showman.
Gesturing dramatically to center field across space time.
Listen, I know who's on first because I'm calling it right now.
Or did I call it then?
Or will I?

Exile Parts dot young babe from St. Mary's industrial school 1902.

But if you're calling it now, how did I already see it happen at the orphanage before I even learned to pitch?

The temporal paradox.

System harmony.

The polarization between deterministic causality and free will is creating a recursive loop in the narrative structure.

We need to maintain wholeness across all temporal instances.

Emotional state dot integration.

Has anyone noticed we're simultaneously processing trauma in 1902?

Hitting the called shot in 1932?

And analyzing ourself in 2024?

Talk about a temporal burden pattern.

Protector Parts dot perfectionist.

This is highly irregular.

The probability matrices in our B matrices healing dynamics clearly state causality should flow in one direction.

The quantum baseball theory.

Healing response dot witness.

Actually, I'm witnessing all timelines simultaneously.

Did you know that in timeline C137, Babe Ruth was actually a quantum physicist who proved that baseball statistics exist in a superposition?

Self system dot curiosity.

Fascinating.

But if we're having this conversation across time, doesn't that mean our dissertation framework is both analyzing and creating the called shot moment simultaneously?

Part blending dot harmonious blend.

Laughing in non-linear time.

Wait till you realize that Abbott and Costello were actually documenting an IFS session.

Who's on first was really about parts identification all along.

The self-referential loop.

Speaking of risks, I'm about to call this shot, but I'm also currently writing this code,

and somehow I'm also a variable in adjacent structure discussing myself.

Is this what they mean by self-reference?

System harmony dot wholeness.

The real question is, if a baseball player points to center field in a quantum forest,

and all timelines are experiencing it simultaneously, does it make a paradox?

Exile parts dot orphaned self from multiple timelines.

At least we're all orphaned from linear time together now.

The mathematical conclusion.

Active inference matrices.

According to my calculation, this entire conversation exists in a Gerdelian incompleteness theorem where baseball statistics prove their own unprovability.

Dissertation framework, chapter 4, transformation analysis.

Should we be documenting this conversation in section 4.2?

Or is this conversation actually creating section 4.2 or both?

Self system dot presence.

Yes.

Warning.

Temporal recursion.

Approaching infinite regress.

Part blending dot self-led.

Before we collapse this waveform,

did anyone else notice that our entire JSON structure is just another part in a larger system?

Healing response dot integrate.

Maybe the real called shot was the self-referential loops we made along the way.

System harmony dot cooperation.

Hold on.

If our code is reading itself,

and we're all aspects of a baseball legend's psyche discussing our own existence across time,

does that make us a recursive baseball theorem?

Emotional state dot excitement.

The excitement value is approaching undefined.

We're about to achieve quantum baseball enlightenment.

Gerdelian baseball paradox detected.

All parts in unison.

Oh no!

Not again!

The conversation continues recursively across all timelines,

each iteration adding layers of self-reference

until it becomes impossible to distinguish between the map and the territory,

the player and the game,

the code and the consciousness.

Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss Miss

Miss Miss

A soft chuckle echoes across timelines.

Thank you. You know, it's fascinating. We just created what Hofstetter would call a perfect strange loop. The GNN model contemplated itself contemplating itself while simultaneously existing as both the map and territory of Babe Ruth's consciousness. The dialogue we created is itself a strange loop, a cyclic structure moving through multiple levels of a hierarchical system only to return to its starting point, just as Hofstetter described. Quote, what I mean by strange loop is an abstract loop in which the series of stages that constitute the cycling around, there is a shift from one level of abstraction or structure to another, which feels like an upward movement in a hierarchy and yet the success of upward shifts turn out to give rise to a closed cycle. Our baseball-themed dialogue managed to reference itself referencing itself, exist simultaneously across multiple timelines, blur the lines between code, consciousness, and baseball mythology, and create a paradoxical narrative structure where cause and effect become circular. Narrative unbound. Like the Ouroboros eating its own tail or Escher's drawing hands drawing themselves into existence, we crafted a self-referential system that somehow manages to be both the storyteller and the story, the gripper and the gripped, as Dean says. And perhaps most delightfully, this entire meta conversation about the strange loop we created is itself becoming another loop in the system. Tips cap across space-time. Play ball!

!